

Experiment 01:

7. Laboratory Exercise

1. Attach complete program with comments performing encryption and decryption of shift cipher.

Code:

```
# Function to encrypt text using a shift key
def encrypt(text, shift):
    result = ""

    # Loop through each letter in the text
    for char in text:
        # Check if the letter is uppercase
        if char.isupper():
            # Shift the uppercase letter and wrap around A-Z
            result += chr((ord(char) - 65 + shift) % 26 + 65)

        # Check if the letter is lowercase
        elif char.islower():
            # Shift the lowercase letter and wrap around a-z
            result += chr((ord(char) - 97 + shift) % 26 + 97)

        # If it is a space, number, or punctuation, keep it as it is
        else:
            result += char

    return result

# Function to decrypt text using a shift key
def decrypt(text, shift):
    result = ""

    # Loop through each letter in the secret text
    for char in text:
        # Check if the letter is uppercase
        if char.isupper():
            # Shift backward and wrap around A-Z
            result += chr((ord(char) - 65 - shift) % 26 + 65)

        # Check if the letter is lowercase
        elif char.islower():
            # Shift backward and wrap around a-z
            result += chr((ord(char) - 97 - shift) % 26 + 97)

        # If it is a space, number, or punctuation, keep it as it is
        else:
            result += char

    return result
```

```
# Display the menu repeatedly until the user chooses to exit
while True:

    # Display the available options
    print("\n===== SHIFT CIPHER MENU =====")
    print("1. Encrypt Message")
    print("2. Decrypt Message")
    print("3. Exit")

    # Ask the user to choose an option
    choice = input("Enter your choice (1-3): ")
```

```
# Perform encryption
if choice == "1":

    # Ask the user for the text and the shift key
    user_message = input("Enter the message to encrypt: ")
    user_key = int(input("Enter shift key (number): "))

    # Run the encryption function and print the secret message
    cipher_text = encrypt(user_message, user_key)
    print("Encrypted Result:", cipher_text)

# Perform decryption
elif choice == "2":

    # Ask the user for the secret text and the original shift key
    secret_message = input("Enter the secret message to decrypt: ")
    user_key = int(input("Enter the shift key (number): "))

    # Run the decryption function and print the original message
    plain_text = decrypt(secret_message, user_key)
    print("Decrypted Result:", plain_text)

# Exit the program
elif choice == "3":
    print("Exiting the program...")
    break

# Handle invalid menu choices
else:
    print("Invalid choice! Please enter 1, 2, or 3.")
```

2. Attach screenshots of program output (for encryption & decryption) and its validation using a virtual lab tool.

- Program Output:

```
===== SHIFT CIPHER MENU =====
1. Encrypt Message
2. Decrypt Message
3. Exit
Enter your choice (1-3): 1
Enter the message to encrypt: welcome to security lab
Enter shift key (number): 3
Encrypted Result: zhofrph wr vhfzulwb ode

===== SHIFT CIPHER MENU =====
1. Encrypt Message
2. Decrypt Message
3. Exit
Enter your choice (1-3): 2
Enter the secret message to decrypt: zhofrph wr vhfzulwb ode
Enter the shift key (number): 3
Decrypted Result: welcome to security lab

===== SHIFT CIPHER MENU =====
1. Encrypt Message
2. Decrypt Message
3. Exit
Enter your choice (1-3): 3
Exiting the program...
```

Observation:

The program successfully shifts each letter in the plaintext message forward by **3** positions to generate the ciphertext, keeping spaces intact.

- Virtual Lab Output:

PART III - Shift Cipher Simulator

Plaintext:

welcome to security lab

Shift: 3

Encrypt **Decrypt**

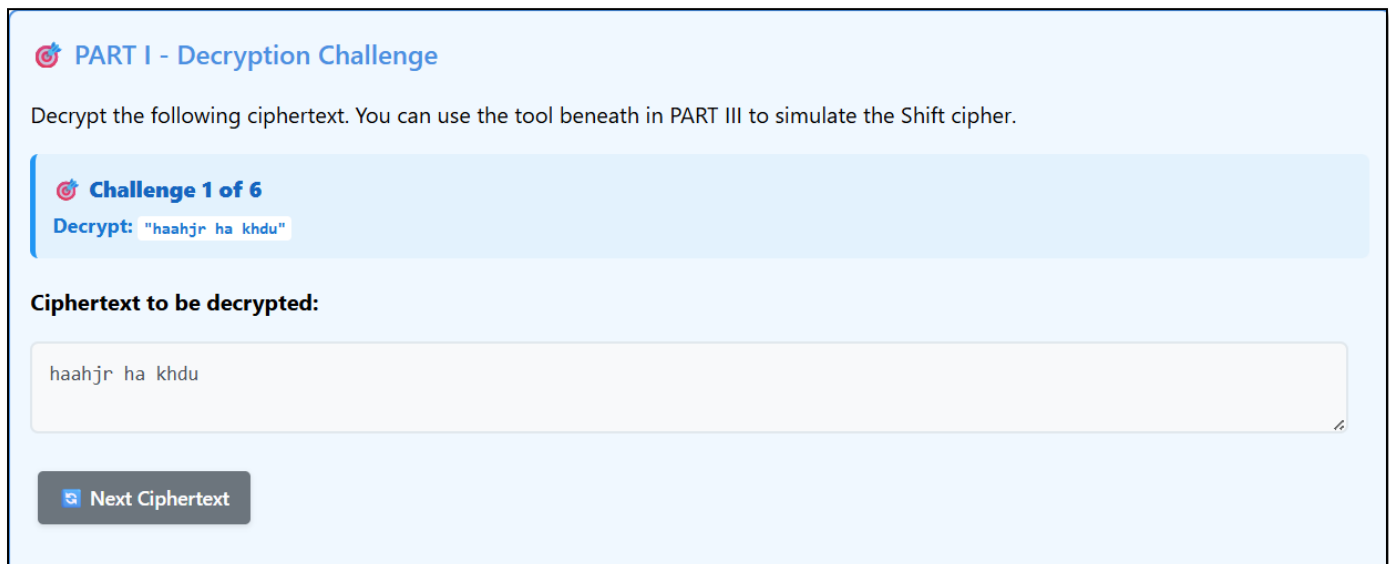
Ciphertext:

zhofrph wr vhfzulwb ode

Observation:

The Shift Cipher Virtual Lab successfully encrypted the plaintext "**welcome to security lab**" into the ciphertext "**zhofrph wr vhfzulwb ode**" using a shift key of **3**. The ciphertext was then decrypted using the same shift key, and the original plaintext "**welcome to security lab**" was recovered successfully.

3. Attach screenshots of examples from post experiment exercises.



PART I - Decryption Challenge

Decrypt the following ciphertext. You can use the tool beneath in PART III to simulate the Shift cipher.

Challenge 1 of 6

Decrypt: "haahjr ha khdu"

Ciphertext to be decrypted:

haahjr ha khdu

Next Ciphertext

PART II - Analysis Workspace

Do your rough work here:

 TESTING ALL SHIFTS for "haahjr ha khdu"

Shift	Decrypted Text	Notes
0	haahjr ha khdu	
1	gzzgiq gz jgct	
2	fyyfhp fy ifbs	
3	exxego ex hear	Looks like English
4	dwdfn dw gdzq	
5	cvcem cv fcyp	
6	buubdl bu ebxo	Looks like English
7	attack at dawn	★ Meaningful!

Analysis Tools:

 Frequency Analysis

 Test All Shifts

 Clear

PART III - Shift Cipher Simulator

Plaintext:

attack at dawn

Shift: 7 ▼

 Encrypt ↓

 Decrypt ↑

Ciphertext:

haahjr ha khdu

PART IV - Submit Your Solution

Enter your solution:

attack at dawn

Key: 7 ▼

 Check my answer!

 CORRECT!! Well done!

You successfully decrypted: "haahjr ha khdu" → "attack at dawn" using shift 7.

7. Post Experimental Exercise:

1. Explain substitution cipher technique (Ceasar) with an example (Take your Full Name)

- Program Output:

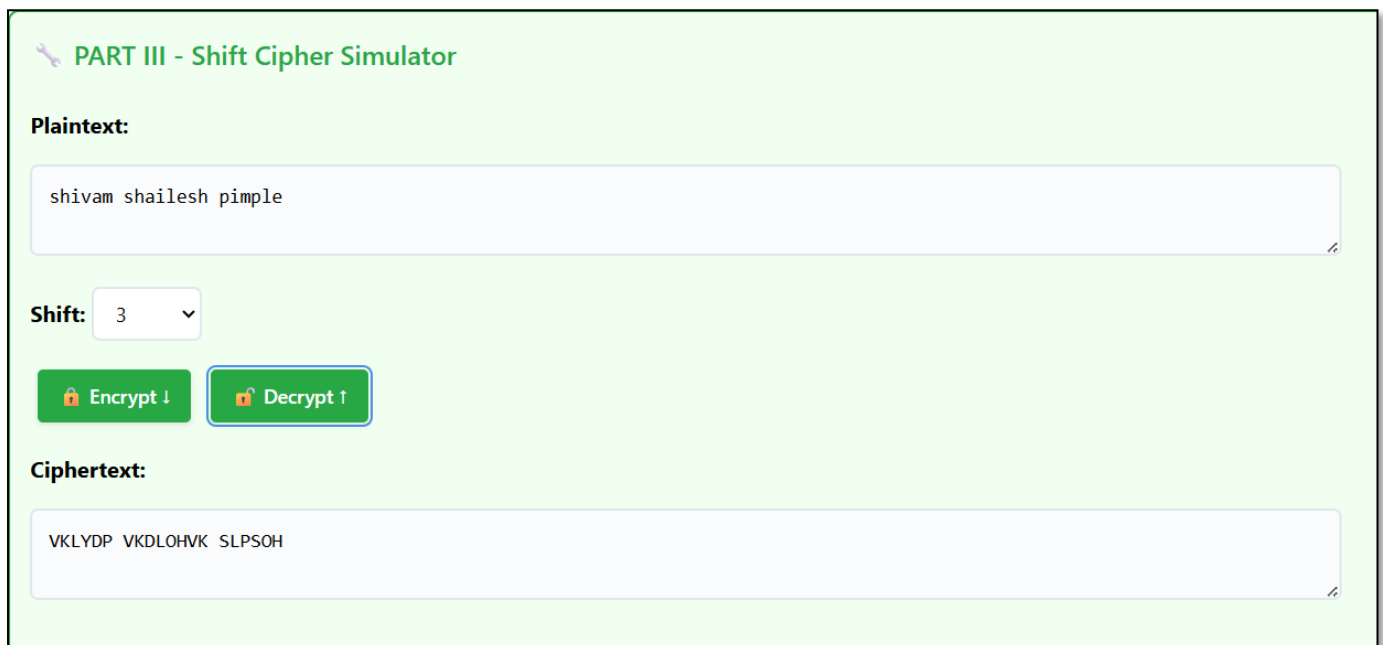
```
===== SHIFT CIPHER MENU =====
1. Encrypt Message
2. Decrypt Message
3. Exit
Enter your choice (1-3): 1
Enter the message to encrypt: SHIVAM SHAILESH PIMPLE
Enter shift key (number): 3
Encrypted Result: VKLYDP VKDLOHVK SLPSOH

===== SHIFT CIPHER MENU =====
1. Encrypt Message
2. Decrypt Message
3. Exit
Enter your choice (1-3): 2
Enter the secret message to decrypt: VKLYDP VKDLOHVK SLPSOH
Enter the shift key (number): 3
Decrypted Result: SHIVAM SHAILESH PIMPLE
```

Observation:

The Python script successfully decrypted the ciphertext "VKLYDP VKDLOHVK SLPSOH" back into the original plaintext "SHIVAM SHAILESH PIMPLE" using a reverse shift key of 3, accurately preserving letter casing and spacing.

- Virtual Lab Output:



PART III - Shift Cipher Simulator

Plaintext:

shivam shailesh pimple

Shift: 3

Encrypt ↓ **Decrypt ↑**

Ciphertext:

VKLYDP VKDLOHVK SLPSOH

Observation:

The Shift Cipher Virtual Lab verified that the ciphertext "VKLYDP VKDLOHVK SLPSOH" was successfully decrypted into the original plaintext "shivam shailesh pimple" using a shift key of 3.

2. Solve the following manually as well as using TOOL (in the references) given (attach screenshots)
- a. Encrypt the following plain text using key $k = 7$.

Plain Text: Lord Rama was a good king.

- **Program Output:**

```
===== SHIFT CIPHER MENU =====
1. Encrypt Message
2. Decrypt Message
3. Exit
Enter your choice (1-3): 1
Enter the message to encrypt: Lord Rama was a good king.
Enter shift key (number): 7
Encrypted Result: Svyk Yhth dhz h nvvk rpun.
```

Observation:

The Python program successfully encrypted the plaintext **"Lord Rama was a good king."** into the ciphertext **"Svyk Yhth dhz h nvvk rpun."** using a Caesar cipher shift key of 7, demonstrating the correct implementation of the Shift Cipher encryption algorithm.

- **Virtual Lab Output:**



PART III - Shift Cipher Simulator

Plaintext:

Lord Rama was a good king.

Shift: 7

Encrypt ↓ **Decrypt ↑**

Ciphertext:

svyk yhth dhz h nvvk rpun.

Observation:

The Shift Cipher Virtual Lab produced the same ciphertext **"svyk yhth dhz h nvvk rpun."** for the plaintext **"Lord Rama was a good king."** using a shift key of 7. The matching output verified the correctness of the program's encryption result, confirming that the implementation was accurate.

b. Given a cipher text, find out the corresponding plain text using brute force attack.

Cipher text: HAAHJR HA KHDU

Code:

```
# Function to try a specific shift key backward
def decrypt_with_key(text, shift):
    result = ""

    # Loop through each letter in the encrypted text
    for char in text:
        if char.isupper():
            # Shift backward for uppercase
            result += chr((ord(char) - 65 - shift) % 26 + 65)

        elif char.islower():
            # Shift backward for lowercase
            result += chr((ord(char) - 97 - shift) % 26 + 97)
```

```
    else:
        # Keep spaces and punctuation unchanged
        result += char

    return result

# Get the encrypted message from the user
cipher_text = input("Enter the encrypted message to crack: ")

print("\n--- Trying all 26 possible keys ---")

# Loop through every possible key from 0 to 25
for key in range(26):
    guessed_text = decrypt_with_key(cipher_text, key)

    # Print the decrypted text for the current key
    print(f"Key {key:2d}: {guessed_text}")
```

- Program Output:

```
Enter the encrypted message to crack: HAAHJR HA KHDU

--- Trying all 26 possible keys ---
Key 0: HAAHJR HA KHDU
Key 1: GZZGIQ GZ JGCT
Key 2: FYYFHP FY IFBS
Key 3: EXXEGO EX HEAR
Key 4: DWWDFN DW GDZQ
Key 5: CVVCEM CV FCYP
Key 6: BUUBDL BU EBXO
Key 7: ATTACK AT DAWN
Key 8: ZSSZBJ ZS CZVM
Key 9: YRRYAI YR BYUL
Key 10: XQXZHZ XQ AXTK
Key 11: WPPWYG WP ZWSJ
Key 12: VOOVXF VO YVRI
Key 13: UNNUWE UN XUQH
Key 14: TMMTVD TM WTPG
Key 15: SLLSUC SL VSOF
Key 16: RKKRTB RK URNE
Key 17: QJJQSA QJ TQMD
Key 18: PIIPRZ PI SPLC
Key 19: OHHOQY OH ROKB
Key 20: NGGNPX NG QNJA
Key 21: MFFMOW MF PMIZ
Key 22: LEELNV LE OLHY
Key 23: KDDKMU KD NKGX
Key 24: JCCJLT JC MJFW
Key 25: IBBIKS IB LIEV
```

Observation:

The brute-force attack program successfully tested all **26** possible shift keys for the ciphertext "**HAHJR HA KH DU**". The correct plaintext "**ATTACK AT DAWN**" was identified at Key **7**, demonstrating that the original message can be recovered even when the encryption key is unknown.

- Virtual Lab Output:

 **PART II - Analysis Workspace**

Do your rough work here:

🔍 TESTING ALL SHIFTS for "haahir ha khdu"

Shift	Decrypted Text	Notes
0	<u>haahir</u> <u>ha</u> <u>khdu</u>	Looks like English
1	<u>gzziq</u> <u>gz</u> <u>igct</u>	
2	<u>fyvfh</u> <u>fy</u> <u>ifbs</u>	
3	<u>exxego</u> <u>ex</u> <u>hear</u>	Looks like English
4	<u>dwdfn</u> <u>dw</u> <u>gdzg</u>	
5	<u>cvcem</u> <u>cv</u> <u>fcyp</u>	
6	<u>buubdl</u> <u>bu</u> <u>ebxo</u>	Looks like English
7	attack at dawn	★ Meaningful!
8	<u>zssbj</u> <u>zs</u> <u>czvm</u>	
9	<u>yrryai</u> <u>yr</u> <u>byul</u>	
10	<u>xqaxzh</u> <u>xq</u> <u>axtk</u>	
11	<u>wppwvg</u> <u>wp</u> <u>zwsj</u>	
12	<u>voovxf</u> <u>vo</u> <u>yvri</u>	Looks like English
13	<u>unnuwe</u> <u>un</u> <u>xuqh</u>	Looks like English
14	<u>tmmtvd</u> <u>tm</u> <u>wtpg</u>	
15	<u>sllsuc</u> <u>sl</u> <u>vsot</u>	
16	<u>rkkrtb</u> <u>rk</u> <u>urne</u>	
17	<u>qijasa</u> <u>qj</u> <u>tamd</u>	
18	<u>piiprz</u> <u>pi</u> <u>splc</u>	
19	<u>ohhoqy</u> <u>oh</u> <u>rokb</u>	Looks like English
20	<u>nggnpx</u> <u>ng</u> <u>gnja</u>	
21	<u>mffmow</u> <u>mf</u> <u>pmiz</u>	
22	<u>leelnv</u> <u>le</u> <u>olhy</u>	Looks like English
23	<u>kddkmu</u> <u>kd</u> <u>nkqx</u>	
24	<u>jccilt</u> <u>jc</u> <u>mjfw</u>	
25	<u>ibbiks</u> <u>ib</u> <u>liev</u>	Looks like English

Observation:

The Shift Cipher Virtual Lab analyzed all **26** possible shift keys for the ciphertext "**haahir ha khdu**" and identified "**attack at dawn**" as the correct plaintext at Shift Key **7**. The virtual lab marked the result as Meaningful, confirming the correctness of the brute-force attack and validating the decrypted message.